LSIE-MLF

Technical Specification

Engineering Blueprint for Autonomous AI Implementation and Enterprise Deployment

Live Stream Inference Engine — Machine Learning Framework

Classification:	**CONFIDENTIAL**
Hardening Baseline:	15-Rule Production Audit
Date:	2026-03-12

Contents

0 Document control and canonical terminology **3**
- 0.1 Document purpose . . . 3
- 0.2 Normative language . . . 3
- 0.3 Canonical terminology registry . . . 3
- 0.4 Version history . . . 4

1 System philosophy and operational scope **4**

2 Deterministic system data flow pipeline **5**

3 Codebase architecture and directory mapping **7**
- 3.1 Directory hierarchy . . . 7
- 3.2 Build context rules . . . 8

4 Core architecture modules **8**
- 4.A Module A — Hardware & transport . . . 8
- 4.B Module B — Ground truth ingestion . . . 10
- 4.C Module C — Orchestration & synchronization . . . 11
- 4.D Module D — Multimodal ML processing . . . 12
- 4.E Module E — Experimentation & analytics . . . 13
- 4.F Module F — Context enrichment . . . 14

5 Data governance and security **15**
- 5.1 Ephemeral vault specification . . . 15
- 5.2 Data classification . . . 15

6 Interface contracts and payload schemas **16**
- 6.1 InferenceHandoffPayload — JSON schema draft 07 . . . 16

7 Mathematical specifications (AU12) **17**
- 7.1 Variable dictionary . . . 17
- 7.2 Step 1: Topological landmark extraction . . . 17
- 7.3 Step 2: Derivation of interocular distance (IOD) . . . 17
- 7.4 Step 3: Au12 distance and baseline calibration . . . 18
- 7.5 Python reference implementation (corrected) . . . 19

8 Deterministic LLM prompt specification **20**
- 8.1 Inference parameter constraints . . . 20
- 8.2 Expected output schema . . . 20
- 8.3 System prompt (canonical) . . . 20

9 Docker compose runtime topology **21**
- 9.1 Container topology . . . 21
- 9.2 Volume mounts . . . 21
- 9.3 GPU allocation . . . 22
- 9.4 Device passthrough . . . 22
- 9.5 Network configuration . . . 22
- 9.6 Service dependency order . . . 22

10 Dependency and runtime matrix **22**
10.1 System requirements 22
10.2 Python package pinning 23

11 Variable extraction matrix **23**

12 Error handling and recovery specification **24**
12.1 Network disconnection 25
12.2 Hardware device loss 25
12.3 Worker process crash 25
12.4 Queue overload 26

13 Autonomous implementation audit checklist **26**

0. Document control and canonical terminology

0.1 Document purpose

This document is the single authoritative specification for the Live Stream Inference Engine — Machine Learning Framework (LSIE-MLF). A developer or autonomous AI agent is required to build the complete LSIE-MLF system using only this document. No external documentation is assumed or required.

0.2 Normative language

Throughout this specification, the following terms carry precise normative meaning:

Term	Meaning
MUST / SHALL / IS REQUIRED TO	The requirement is absolute and non-negotiable.
MUST NOT / SHALL NOT	The prohibition is absolute.
IS DEFINED AS	The term has a single, unambiguous meaning as stated.

0.3 Canonical terminology registry

Every system component has exactly one canonical name. All prior synonyms are retired. Implementations MUST use only the canonical identifiers listed below.

Canonical name	Retired synonyms	Definition
API Server	api, FastAPI server, web server, ASGI server	The FastAPI application process that serves REST endpoints on port 8000. Container name: api.
ML Worker	worker, Celery node, Celery worker, GPU worker, inference worker	The Celery consumer process that executes all GPU-bound ML inference tasks. Container name: worker.
Message Broker	Redis, queue, task queue, broker	The Redis in-memory data store that brokers Celery task queues between the API Server and the ML Worker. Container name: redis.
Persistent Store	postgres, PostgreSQL, database, relational database, DB	The PostgreSQL relational database that persists analytical metrics and experiment state. Container name: postgres.
Capture Container	stream_scrcpy, scrcpy container, capture service, stream ingester	The privileged Docker container responsible for physical USB I/O and raw audio-visual stream extraction. Container name: stream_scrcpy.
IPC Pipe	FIFO, named pipe, POSIX pipe, audio pipe, kernel pipe	The POSIX Named Pipe located at /tmp/ipc/audio_stream.raw used for zero-copy data transfer between the Capture Container and the ML Worker.
Ephemeral Vault	24-hour vault, data vault, transient storage, secure buffer	The AES-256-GCM encrypted transient storage regime for raw media buffers, subject to mandatory 24-hour secure deletion.

Canonical name	Retired synonyms	Definition
InferenceHandoffPayload	handoff schema, payload schema, inference payload	The JSON Schema Draft 07 contract governing data exchange between Module C and Module D, and from Module D to Module E.

0.4 Version history

Version	Date	Description
v1.0	2026-03-12	Initial unified specification.
v2.0	2026-03-12	Production-hardened. Applied 15-rule deterministic audit: canonical terminology, module contracts, data-flow pipeline, dependency pinning, AU12 Python fix, IPC lifecycle, temporal sync formulas, Ephemeral Vault hardening, Docker topology expansion, LLM determinism constraints, error-handling matrices, variable matrix expansion, and autonomous implementation audit.

1. System philosophy and operational scope

The deployment of the LSIE-MLF requires a flawless convergence of real-time computer vision, asynchronous stream processing, and natural language processing. This document is the Unified Final Technical Specification (v2.0), expressly designed to merge the structural clarity and operational boundary rules of the legacy v4.0 architecture with the deterministic, hardware-aligned implementation details of the LSIE-MLF executable draft.

The overarching objective of this specification is to provide a single, authoritative source of truth that is exhaustively detailed for human systems architects, while remaining strictly structured and deterministic for interpretation by autonomous AI coding agents.

The operational scope of the LSIE-MLF mandates the capacity to ingest raw, multimodal data streams from mobile hardware and execute concurrent, high-throughput heuristic evaluations without succumbing to inter-process communication bottlenecks or resource starvation.

In live-stream environments, data is inherently chaotic: audio streams arrive in uncompressed pulse-code modulation (PCM) formats, video frames are subject to lighting and head-pose variations, and digital metadata (such as chat WebSockets) operates on independent network clocks. To process this multimodal input into structured, actionable intelligence, the system strictly enforces a schema-first data exchange paradigm, a modular boundary constraint system, and absolute dependency determinism.

Every layer of the architecture — from the physical USB transport utilizing Android Debug Bridge (ADB) protocols to the execution of INT8-quantized transformer models — MUST behave predictably. The system is bound by rigorous data governance protocols. Because the machine learning pipelines extract high-fidelity biometric data (facial landmarks) and process raw human speech, the architecture implements the Ephemeral Vault policy. This policy ensures that the massive volumes of sensitive data generated by the inference engine remain compliant with data minimization principles, the General Data Protection Regulation (GDPR), and the Health Insurance Portability and Accountability Act (HIPAA), strictly prohibiting the persistent storage of raw, personally identifiable media.

2. Deterministic system data flow pipeline

This section defines the complete end-to-end data path through the LSIE-MLF system. Every transition specifies the transport mechanism, payload format, synchronization mechanism, and error behavior.

1. Mobile hardware → capture container

Transport	USB passthrough via /dev/bus/usb device mapping into the Capture Container; scrcpy v2.x raw mirroring protocol.
Payload	Audio: Uncompressed PCM signed 16-bit little-endian (s16le), 48 kHz mono. Video: Raw H.264 frames via scrcpy decoder.
Sync	Android hardware clock polled via adb shell 'echo $EPOCHREALTIME' every 30 seconds; drift offset applied to timestamps.
Error behavior	If USB connection is lost, the Capture Container logs the event, halts pipe writes, and polls reconnection every 2 seconds for 60 seconds before graceful termination.

2. Capture container → ML worker

Transport	IPC Pipe at /tmp/ipc/audio_stream.raw via shared Docker volume ipc-share.
Payload	Continuous byte stream of PCM s16le audio at 48 kHz.
Sync	Non-blocking file descriptor opened using exec 3<> /tmp/ipc/audio_stream.raw.
Error behavior	If the ML Worker crashes, pipe writes return EAGAIN when buffer fills. Overflowed bytes are silently discarded until worker restart.

3. Ml worker internal: audio resampling

Transport	FFmpeg subprocess pipeline using stdin/stdout pipes.
Payload	Input: PCM s16le 48 kHz. Output: PCM s16le 16 kHz via pipe:1.
Sync	FFmpeg executes synchronously with the audio processing thread ensuring ordered delivery.
Error behavior	If FFmpeg exits unexpectedly, the worker detects the failure and restarts the subprocess within 1 second.

4. Module b → module c

Transport	Python in-process function calls inside the ML Worker event loop.
Payload	Python dictionaries containing uniqueId, event_type, timestamp_utc, and payload fields.
Sync	Events timestamped using host UTC and drift-corrected offset before entering the deque ring buffer.
Error behavior	If the WebSocket disconnects, exponential backoff reconnection is attempted (1s initial delay, 30s max, 10 retries).

5. Module c → module d

Transport	In-process Python function call process_segment().
Payload	InferenceHandoffPayload JSON object validated against the Pydantic schema.
Sync	Segment windows fixed at 30 seconds and validated before inference.
Error behavior	If schema validation fails, the segment is logged and discarded while processing continues.

6. Module d → module e

Transport	Celery asynchronous task dispatch through the Message Broker.
Payload	JSON payload containing session_id, segment_id, AU12 intensity, transcription, semantic match, pitch, jitter, and shimmer.
Sync	Redis append-only persistence ensures tasks survive broker restarts.
Error behavior	If the broker is unreachable Celery retries with exponential backoff up to five attempts before writing failed payloads to /data/processed/failed_tasks/.

7. Module e → persistent store

Transport	psycopg2-binary PostgreSQL connection pool.
Payload	SQL INSERT statements with parameterized queries storing metrics as DOUBLE PRECISION and timestamps as TIMESTAMPTZ.
Sync	SERIALIZABLE isolation for experiment updates and READ COMMITTED for metric inserts.
Error behavior	If the database is unreachable the module buffers up to 1000 records in memory and retries every 5 seconds before overflowing to CSV.

3. Codebase architecture and directory mapping

The architectural foundation of the LSIE-MLF dictates the efficiency of ongoing development, continuous integration, and containerized deployment. To support the simultaneous execution of I/O-bound APIs and heavily compute-bound GPU inference models, the system adopts a strictly segmented monorepo architecture.

A monorepo strategy ensures that shared utilities, core mathematical libraries, and interface contracts are natively accessible to all independent microservices without the operational overhead of maintaining complex internal package registries or private PyPI servers. However, containerizing applications within a monorepo introduces specific continuous integration challenges. If a developer alters a lightweight web API service, the deployment pipeline MUST NOT be forced to recompile the heavy ML Worker images, which contain massive CUDA toolchains.

To mitigate this risk, the codebase architecture follows a modular service pattern where individual computational domains are physically isolated within the directory structure but logically unified through a root-level orchestrator. The architecture fundamentally relies on FastAPI for web routing and Celery for asynchronous task management. Because FastAPI operates on a single-threaded asynchronous event loop, it is highly susceptible to blocking if tasked with heavy computational workloads. Therefore, the directory structure enforces a hard physical separation between the API Server application logic and the ML Worker logic.

3.1 Directory hierarchy

Path	Purpose
`/docker-compose.yml`	The root orchestrator defining the multi-container topology, virtual networks, GPU device reservations, and shared IPC volume mounts. This file is the single entry point for all container lifecycle operations.
`/services/api/`	Houses the API Server logic. Contains the ASGI entry point (Uvicorn), RESTful routing controllers, and dependency injection modules for the Persistent Store connection pool.

Path	Purpose
`/services/worker/`	Contains the ML Worker Celery task consumers. Houses the orchestration logic that coordinates discrete ML pipeline steps, pulling GPU-bound inference jobs from the Message Broker.
`/services/stream_ingest/`	Dedicated to the Capture Container's raw audio-visual extraction mechanisms. Contains the shell orchestrators responsible for instantiating the IPC Pipe and executing the ADB screen mirroring and audio extraction protocols.
`/packages/ml_core/`	Houses the shared mathematical and inference utilities utilized across the stack. This includes the Python reference implementations for topological facial landmark extraction, Action Unit mathematics, and audio transcription wrappers.
`/packages/schemas/`	Centralizes Pydantic models and JSON Schema Draft 07 definitions ensuring strict interface contracts and data validation rules across services.
`/data/raw/`	Local host volume mount for transient staging of extracted audio-visual buffers including IPC pipe buffers and raw PCM streams. Mounted as tmpfs in production.
`/data/interim/`	Storage for intermediate processing states such as downsampled audio buffers and cropped video frames for localized neural network analysis.
`/data/processed/`	Destination for validated ML inference outputs and structured analytical JSON payloads prior to database ingestion.
`/requirements/`	Maintains deterministic dependency tracking separating base requirements, API dependencies, and heavy ML dependencies for optimized Docker layer caching.

3.2 Build context rules

All Dockerfiles located in /services directories MUST set their build context to the monorepo root so that shared /packages directories are available during build. Strict .dockerignore rules MUST prevent unnecessary assets from entering images. The API Server container MUST exclude heavy ML dependencies, while the ML Worker containers bypass unnecessary web assets.

4. Core architecture modules

The modules defined below form the operational spine of the LSIE-MLF specification. Each module contains explicit execution details followed by a formal contract defining inputs, outputs, dependencies, side effects, and failure modes.

4.A Module A — Hardware & transport

Purpose: Extract uncompressed, zero-latency audio and video streams from tethered mobile hardware and transport them into the containerized environment without network serialization overhead.

Technology stack: scrcpy v2.x, IPC Pipe (POSIX Named Pipe), Bash file descriptor manipulation, Docker shared volume ipc-share

4.A.1 Execution details

Module A interfaces physically with mobile devices via USB protocols. The Capture Container maps the host /dev/bus/usb subsystem directly into the container allowing the containerized runtime to communicate with hardware.

Traditional Android mirroring encodes streams into H.264 or H.265 introducing 35–70 ms of latency through Android SurfaceFlinger and MediaProjection subsystems which is unacceptable for real-time ML pipelines.

To extract audio with minimal latency the scrcpy service is executed using specific flags that force raw PCM output and limit buffering.

The architecture avoids REST APIs or Redis queues for audio transport. Instead a shared Docker volume mounts /tmp/ipc across containers enabling POSIX IPC pipes for zero-copy streaming.

- –audio-codec=raw — bypasses AAC or Opus compression and forces raw PCM output.
- –audio-buffer=30 — limits the internal scrcpy audio buffer to 30 ms.
- –audio-dup with –no-audio-playback — mirrors audio to the container while preserving local playback.

IPC pipe lifecycle specification

1. **Creation:** Capture Container entrypoint executes mkfifo /tmp/ipc/audio_stream.raw before audio capture. If the pipe exists it is removed and recreated.
2. **Non-blocking open:** Capture Container opens the pipe with exec 3<> /tmp/ipc/audio_stream.raw to decouple reader and writer lifecycle states.
3. **Writer role:** scrcpy outputs raw audio to stdout which is piped to the named pipe using dd writing to file descriptor 3.
4. **Reader role:** ML Worker reads /tmp/ipc/audio_stream.raw when its FFmpeg subprocess initializes.
5. **Shutdown:** On graceful shutdown the Capture Container closes descriptor 3 and deletes the pipe file.
6. **Crash recovery:** If the ML Worker crashes the pipe buffer fills and writes return EAGAIN. Overflowed audio is discarded until the worker reconnects.

Module A — Formal contract

Module A Contract

Inputs
USB-connected Android device producing raw PCM audio (s16le 48 kHz) and H.264 video via scrcpy.

Outputs
Continuous PCM audio stream written to /tmp/ipc/audio_stream.raw and raw video frames through the scrcpy video sink.

Dependencies
scrcpy v2.x, ADB server, /dev/bus/usb passthrough, Docker volume ipc-share mounted at /tmp/ipc.

Side effects
Creates named pipe /tmp/ipc/audio_stream.raw and opens file descriptor 3 inside the container.

Failure modes
USB disconnection triggers 2-second polling reconnection loop for 60 seconds. Pipe overflow discards data silently. Permission denial on /dev/bus/usb prevents container start.

4.B Module B — Ground truth ingestion

Purpose: Ingest real-time contextual metadata, chat streams, and user interactions to create ground-truth datasets.

Technology stack: Python 3.11.x, TikTokLive connector, EulerStream signature API

4.B.1 Execution details

The system continuously ingests live chat and interaction events from the platform using the TikTokLive Python connector.

Connection requires ByteDance anti-crawler signatures including X-Bogus and msToken. Signature generation is delegated to the EulerStream signature server.

Once connected the ingestion module enforces the Action_Combo constraint where multiple events must coincide temporally to form a valid ground truth signal.

Incoming protobuf messages are parsed into Python dictionaries and high-value composite events are staged for later synchronization with inference outputs.

Module B — Formal contract

Module B Contract

Inputs
Authenticated TikTok Webcast WebSocket event stream using EulerStream-signed URL parameters.

Outputs
Parsed dictionaries representing high-value events containing uniqueId, event_type, timestamp_utc, gift_value, and combo_events.

Dependencies
TikTokLive library, EulerStream signature API, network access to TikTok Webcast servers.

Side effects
Maintains persistent WebSocket connection and refreshes signature tokens during reconnects.

Failure modes
WebSocket disconnection triggers exponential backoff reconnect. EulerStream outage results in degraded mode. Malformed protobuf events are logged and discarded.

4.C Module C — Orchestration & synchronization

Purpose: Align timestamps between device hardware clocks, IPC media streams, and external WebSocket events.

Technology stack: Python 3.11.x, collections.deque, FFmpeg static build, ADB

4.C.1 Temporal drift correction

Android hardware clocks drift relative to UTC over time especially during thermal throttling or power state transitions.

Without correction media frames and WebSocket metadata diverge breaking temporal correlation between speech, facial expression, and digital events.

4.C.2 Audio resampling

The ML models require audio sampled at 16 kHz while Android devices output audio at 48 kHz.

A lightweight FFmpeg pipeline continuously resamples audio from 48 kHz to 16 kHz and streams the result to the Python process.

Drift polling specification

Parameter	Value
Polling interval	30 seconds
ADB command	adb shell 'echo $EPOCHREALTIME'
Return format	Decimal epoch with microsecond precision
Maximum tolerated drift	150 milliseconds

Parameter	Value
Drift formula	drift_offset = host_utc_time - android_epoch_time
Timestamp correction	corrected_ts = original_ts + drift_offset
Fallback strategy	Freeze drift_offset after 3 ADB failures; reset to zero after 5 minutes

FFmpeg resampling command

```
ffmpeg -f s16le -ar 48000 -ac 1 -i /tmp/ipc/audio_stream.raw -ar 16000 -f s16le -ac 1 pipe:1
```

Module C — Formal contract

Module C Contract

Inputs
Raw PCM audio from IPC Pipe, ground truth events from Module B, and Android epoch timestamps from ADB.

Outputs
16 kHz resampled audio stream, drift-corrected metadata stored in a deque buffer, and InferenceHandoffPayload segments.

Dependencies
FFmpeg binary, ADB server, readable IPC Pipe, Python 3.11 runtime.

Side effects
Spawns persistent FFmpeg subprocess and maintains in-memory deque buffer.

Failure modes
Missing IPC pipe triggers polling for 30 seconds. FFmpeg crashes restart automatically. ADB failures invoke fallback drift strategy.

4.D Module D — Multimodal ML processing

Purpose: Perform speech transcription, facial landmark inference, acoustic analysis, and semantic evaluation.

Technology stack: faster-whisper v1.2.1, mediapipe v0.10.x, parselmouth v0.4.4, spacy v3.7.x

4.D.1 Speech transcription

Audio segments are transcribed using faster-whisper with the CTranslate2 inference backend.

The engine uses INT8 quantization and requires CUDA 12 and cuDNN 9 for GPU execution.

4.D.2 Computer vision — face mesh

Video frames are processed using MediaPipe Face Mesh generating a 478-vertex 3D landmark mesh.

The model uses Procrustes Analysis to construct a normalized metric face space.

4.D.3 Acoustic analysis

The Praat engine accessed through parselmouth extracts pitch, jitter, and shimmer metrics from speech audio.

4.D.4 Text pre-processing

spaCy performs tokenization, normalization, and linguistic preprocessing prior to LLM semantic evaluation.

Module D — Formal contract

Module D Contract

Inputs
16 kHz audio stream, raw video frames, and InferenceHandoffPayload segments.

Outputs
Speech transcripts, 478-vertex facial landmark arrays, AU12 intensity scores, acoustic metrics, and semantic evaluation JSON results.

Dependencies
faster-whisper, CTranslate2, CUDA ≥12, cuDNN ≥9, mediapipe, parselmouth, spacy, Azure OpenAI API.

Side effects
Loads ML models into GPU memory and performs external HTTPS calls to Azure OpenAI.

Failure modes
GPU OOM triggers worker restart. Whisper load failure aborts startup. Missing face returns null facial metrics. LLM timeout retries once before recording null.

4.E Module E — Experimentation & analytics

Purpose: Aggregate inference metrics, persist results to the database, and run adaptive experimentation logic.

Technology stack: psycopg2-binary v2.9.x, pandas v2.2.x, SciPy, Streamlit

4.E.1 Execution details

High-frequency ML outputs such as AU12 arrays, pitch trajectories, and semantic scores are aggregated into structured time-series datasets.

pandas performs high-performance data manipulation while SciPy provides statistical analysis.

Thompson Sampling is used to dynamically evaluate greeting rules and behavioral prompts.

Operational metrics and experiment state are visualized through a Streamlit dashboard.

Module E — Formal contract

Module E Contract

Inputs
Inference payloads from Module D delivered through the Celery task queue.

Outputs
Time-series records stored in the Persistent Store, Thompson Sampling experiment assignments, and Streamlit dashboards.

Dependencies
psycopg2-binary, pandas, SciPy, Streamlit, Redis Message Broker, PostgreSQL database.

Side effects
Writes metrics to database tables and runs a Streamlit UI service.

Failure modes
Database outage buffers 1000 records before disk overflow. Celery task deserialization errors are logged and discarded.

4.F Module F — Context enrichment

Purpose: Perform asynchronous scraping of external web sources to enrich contextual information about the stream.

Technology stack: Celery v5.4.x, Message Broker, patchright v1.58.2

4.F.1 Execution details

Context enrichment tasks run asynchronously outside the main inference loops.

Celery workers execute scraping jobs that gather external metadata about creators or trending topics.

patchright is used instead of standard Playwright to bypass bot-detection mechanisms by patching Chromium automation fingerprints.

Module F — Formal contract

Module F Contract

Inputs
Celery tasks containing target URL, scrape_type, and timeout_seconds parameters.

Outputs
JSON objects containing scraped metadata including source_url, scraped_at_utc timestamp, and extracted data dictionary.

Dependencies
Celery, Redis Message Broker, patchright browser automation toolkit, Chromium runtime.

Side effects
Launches ephemeral headless browser instances and performs outbound HTTP requests.

Failure modes
CAPTCHA or HTTP blocking returns empty result. Browser crash triggers retry. Timeout terminates browser. Broker outage triggers Celery reconnection.

5. Data governance and security

The LSIE-MLF processes highly sensitive multimodal data including raw speech audio and dense 3D facial topology maps. Operation of such a system requires strict regulatory compliance, strong encryption practices, and strict data minimization policies to ensure that personally identifiable biometric information never persists beyond the transient processing pipeline.

5.1 Ephemeral vault specification

Parameter	Specification
Encryption algorithm	AES-256-GCM (authenticated encryption with associated data)
Encryption library	PyCryptodome (pycryptodome >= 3.20.0)
Key length	256 bits
IV/Nonce length	96 bits (12 bytes) generated using os.urandom(12)
Key generation	os.urandom(32) generating a unique key per debug session
Key storage location	Process memory only. Keys are never written to disk, environment variables, or configuration files.
Key lifecycle	Generated at session start and destroyed when the container terminates. No persistence mechanism exists.
Secure deletion method	shred -vfz -n 3 executed by internal cron every 24 hours on /data/raw/ and /data/interim/
Retention period	Maximum 24 hours for any raw media buffer touching physical disk

5.2 Data classification

Classification	Description	Storage	Retention
Transient Data	Raw PCM audio buffers and uncompressed video frames existing only in volatile memory during processing.	`/tmp/ipc/ (tmpfs) and kernel pipe buffers`	Duration of active processing only with no disk persistence.
Debug Storage	Encrypted diagnostic media buffers written to disk only when algorithmic anomalies occur.	`/data/raw/ and /data/interim/`	Maximum 24 hours followed by secure deletion.
Permanent Analytical Storage	Anonymized analytical metrics including AU12 scores, acoustic metrics, and semantic evaluation results.	`Persistent Store (PostgreSQL)`	Indefinite subject to database backup policies.

The system MUST never store raw facial images, voiceprints, or other personally identifiable biometric media in the Persistent Store. MediaPipe landmarks are reduced to normalized geometric ratios such as AU12 intensity values before persistence, ensuring biometric anonymization.

6. Interface contracts and payload schemas

In distributed multimodal machine learning pipelines, loosely typed interfaces between processing modules can cause catastrophic runtime failures. Incorrect serialization of numeric arrays or omission of required fields can break downstream indexing and database ingestion.

To guarantee deterministic communication between modules, LSIE-MLF enforces strict interface contracts defined using JSON Schema Draft 07 and validated using Pydantic models within the Python runtime.

6.1 InferenceHandoffPayload — JSON schema draft 07

```
{ "$schema": "http://json-schema.org/draft-07/schema#", "title": "InferenceHandoffPayload",
"description": "Standardized schema for multimodal ML pipeline handoff.", "type": "object",
"required": ["session_id", "timestamp_utc", "media_source", "segments"], "properties": {
"session_id": { "type": "string", "pattern":
"^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$", "description": "UUID v4
representing the continuous live stream session." }, "timestamp_utc": { "type": "string",
"format": "date-time", "description": "RFC 3339 UTC timestamp of inference event completion." },
"media_source": { "type": "object", "required": ["stream_url", "codec", "resolution"],
"properties": { "stream_url": { "type": "string", "format": "uri" }, "codec": { "type": "string",
"enum": ["h264", "h265", "raw"] }, "resolution": { "type": "array", "items": { "type": "integer",
"minimum": 1 }, "minItems": 2, "maxItems": 2, "description": "[width, height] in pixels." } } },
"segments": { "type": "array", "description": "Array of analyzed video segments." } } }
```

7. Mathematical specifications (AU12)

The quantitative measurement of human facial affect is governed by the Facial Action Coding System (FACS). Within this taxonomy Action Unit 12 (AU12), the Lip Corner Puller, corresponds to contraction of the zygomaticus major muscle drawing the mouth corners upward and outward.

Module D computes AU12 intensity from the 478 three-dimensional landmarks generated by the MediaPipe Face Mesh model. Because head distance and pose distort raw pixel distances, geometric normalization is performed using the interocular distance (IOD) as a scale-invariant reference computed in full 3D Euclidean space.

7.1 Variable dictionary

Symbol	Code name	Definition
L_i	`landmarks[i]`	3D vector (x_i, y_i, z_i) of MediaPipe landmark i.
L_61	`landmarks[61]`	Left lip corner landmark.
L_291	`landmarks[291]`	Right lip corner landmark.
L_33	`landmarks[33]`	Left eye outer corner.
L_133	`landmarks[133]`	Left eye inner corner.
L_362	`landmarks[362]`	Right eye inner corner.
L_263	`landmarks[263]`	Right eye outer corner.
E_L	`e_l`	Geometric center of left eye midpoint(L_33,L_133).
E_R	`e_r`	Geometric center of right eye midpoint(L_362,L_263).
IOD	`iod`	3D Euclidean distance between E_L and E_R.
D_mouth	`d_mouth`	3D Euclidean distance between lip corners.
$B_neutral$	`self.b_neutral`	Neutral baseline average of D_mouth/IOD.
$I_AU12(t)$	`i_au12`	Normalized AU12 intensity at time t.
$alpha$	`self.alpha`	Empirical linear projection multiplier.
$Score_AU12(t)$	`score_au12`	Final AU12 FACS score in range 0–5.

7.2 Step 1: Topological landmark extraction

The anatomical points required for AU12 computation are extracted from the MediaPipe landmark array.

- Mouth corners: L_61 (left) and L_291 (right)
- Left eye bounds: L_33 outer and L_133 inner
- Right eye bounds: L_362 inner and L_263 outer

7.3 Step 2: Derivation of interocular distance (IOD)

Eye centers are computed as midpoints between inner and outer eye landmarks and used to calculate the normalization scalar.

$$E_L = ((x_{33}+x_{133})/2, (y_{33}+y_{133})/2, (z_{33}+z_{133})/2); \quad E_R = ((x_{362}+x_{263})/2, (y_{362}+y_{263})/2, (z_{362}+z_{263})/2)$$

$$IOD = \sqrt{(E_R.x - E_L.x)^2 + (E_R.y - E_L.y)^2 + (E_R.z - E_L.z)^2}$$

7.4 Step 3: Au12 distance and baseline calibration

The raw AU12 distance equals the Euclidean distance between lip corners.

A neutral baseline B_neutral is computed from the rolling average of D_mouth/IOD during the calibration phase.

$$D_{mouth} = \sqrt{(x_{291} - x_{61})^2 + (y_{291} - y_{61})^2 + (z_{291} - z_{61})^2}$$

$$I_{AU12}(t) = (D_{mouth}(t)/IOD(t)) - B_{neutral}$$

$$Score_{AU12}(t) = max(0, \alpha \cdot I_{AU12}(t))$$

7.5 Python reference implementation (corrected)

```
import numpy as np
from typing import List

EPSILON: float = 1e-6

class AU12Normalizer:
    def __init__(self, alpha: float = 5.0) -> None:
        self.alpha: float = alpha
        self.b_neutral: float | None = None
        self.calibration_buffer: List[float] = []

    def compute_intensity(self, landmarks: np.ndarray, is_calibrating: bool = False) -> float:
        l_61 = landmarks[61]
        l_291 = landmarks[291]
        l_33 = landmarks[33]
        l_133 = landmarks[133]
        l_362 = landmarks[362]
        l_263 = landmarks[263]

        e_l = np.mean([l_33, l_133], axis=0)
        e_r = np.mean([l_362, l_263], axis=0)
        iod = float(np.linalg.norm(e_r - e_l))

        if iod < EPSILON:
            return 0.0

        d_mouth = float(np.linalg.norm(l_291 - l_61))
        raw_ratio = d_mouth / iod

        if is_calibrating:
            self.calibration_buffer.append(raw_ratio)
            self.b_neutral = float(np.mean(self.calibration_buffer))
            return 0.0

        if self.b_neutral is None:
            raise ValueError("Baseline not calibrated.")

        i_au12 = raw_ratio - self.b_neutral
        score_au12 = max(0.0, self.alpha * i_au12)

        return float(min(score_au12, 5.0))
```

v2.0 Corrections Applied

Landmark indexing corrected to use landmarks[i]; epsilon guard added to avoid division by zero when IOD approaches zero; AU12 output hard-clamped to 5.0; full Python type annotations added for deterministic static analysis.

8. Deterministic LLM prompt specification

Module D must determine whether a spoken utterance satisfies a predefined semantic rule such as a required greeting phrase. Traditional string-matching algorithms like Levenshtein distance or BLEU fail because human speech contains fillers, paraphrasing, and transcription noise.

To perform robust evaluation while maintaining reproducibility, LSIE-MLF uses Azure OpenAI Chat Completion models configured in deterministic mode. Temperature is fixed to zero, nucleus sampling is disabled, and structured JSON outputs are enforced through schema constraints.

8.1 Inference parameter constraints

Parameter	Value	Justification
`temperature`	`0.0`	Collapses the probability distribution to deterministic token selection.
`top_p`	`1.0`	Disables nucleus sampling ensuring deterministic token selection.
`max_tokens`	`500`	Ensures adequate space for reasoning plus structured JSON output.
`response_format`	`JSON Schema Structured Outputs`	Forces the model to return machine-parseable JSON rather than conversational text.
`seed`	`42`	Activates Azure OpenAI deterministic inference mode.

8.2 Expected output schema

```
{ "type": "object", "required": ["reasoning","is_match","confidence_score"], "properties": {
"reasoning": { "type": "string" }, "is_match": { "type": "boolean" }, "confidence_score": {
"type": "number", "minimum": 0, "maximum": 1 } }, "additionalProperties": false }
```

8.3 System prompt (canonical)

```
You are a highly precise linguistic evaluation assistant.
Your objective is to determine whether a transcribed 'Actual Utterance' is semantically
equivalent to an 'Expected Greeting Rule'.

Inputs:
1. Expected Greeting Rule
2. Actual Utterance

Evaluation constraints:
- Minor transcription errors and filler words are acceptable if the semantic intent remains
unchanged.
- Major deviations in subject matter must be rejected.

Steps:
1. Produce a reasoning explanation evaluating semantic similarity.
2. Output boolean is_match.
3. Output confidence_score between 0.00 and 1.00.

Return the result strictly as valid JSON without markdown formatting.
```

9. Docker compose runtime topology

The LSIE-MLF runtime environment consists of multiple containerized services orchestrated through Docker Compose. Containers share a common bridge network and specific named volumes enabling high-throughput IPC communication and persistent storage where required.

9.1 Container topology

Container	Role	Image	Port	Net	GPU	Restart
`api`	API Server providing FastAPI REST endpoints.	`python:3.11-slim`	8000:8000	app-network	None	`unless-stopped`
`worker`	ML Worker executing Celery inference tasks.	`nvidia/cuda:12.2.2-cudnn9-runtime-ubuntu22.04`	None	app-network	All (capabilities: [gpu], driver: nvidia)	`on-failure:5`
`redis`	Message Broker used by Celery.	`redis:7-alpine`	6379 (internal only)	app-network	None	`unless-stopped`
`postgres`	Persistent Store for experiment and metric data.	`postgres:16-alpine`	5432 (internal only)	app-network	None	`unless-stopped`
`stream_scrcpy`	Capture Container extracting raw USB audio/video streams.	`ubuntu:22.04 + scrcpy v2.x`	None	app-network	None	`on-failure:3`

9.2 Volume mounts

Volume	Mount path	Containers	Purpose
`ipc-share`	`/tmp/ipc/`	stream_scrcpy, worker	Shared volume containing IPC named pipes.
`data-raw`	`/data/raw/`	worker	Transient debug media buffers.
`data-interim`	`/data/interim/`	worker	Intermediate processing artifacts.
`data-processed`	`/data/processed/`	worker, api	Validated inference outputs and fallback storage.
`pg-data`	`/var/lib/postgresql/data/`	postgres	Persistent PostgreSQL database storage.

9.3 GPU allocation

The ML Worker container receives exclusive GPU access through Docker's deploy.resources reservation syntax and requires CUDA 12 with cuDNN 9 for CTranslate2 compatibility.

```
deploy:
  resources:
    reservations:
      devices:
        - driver: nvidia
          count: all
          capabilities: [gpu]
```

9.4 Device passthrough

USB passthrough allows the capture container to communicate directly with the Android device through the host kernel device subsystem.

```
devices:
  - /dev/bus/usb:/dev/bus/usb
```

9.5 Network configuration

All services run within the same Docker bridge network allowing container name DNS resolution. Only the API service exposes ports externally.

9.6 Service dependency order

1. redis — starts first; health check redis-cli ping.
2. postgres — starts after redis; health check pg_isready.
3. stream_scrcpy — starts after redis to initialize IPC pipe.
4. worker — starts after redis, postgres, and stream_scrcpy.
5. api — starts after worker and postgres.

10. Dependency and runtime matrix

10.1 System requirements

Component	Required ver	Notes
Python	`3.11.x`	All containers MUST run Python 3.11. Python 3.12+ is untested with CTranslate2.
NVIDIA CUDA	⩾ `12.0`	Required for faster-whisper and CTranslate2 runtime compatibility.

Component	Required ver	Notes
cuDNN	⩾ 9.0	Bundled within nvidia/cuda:12.2.2-cudnn9-runtime image.
NVIDIA Container Toolkit	latest	Required for Docker GPU passthrough.
GPU VRAM	⩾ 6GB	Minimum VRAM for faster-whisper large-v3 INT8 plus MediaPipe runtime.
Docker Compose	v2.x	Required for deploy.resources GPU reservation syntax.

10.2 Python package pinning

Package	Version	Container(s)	Purpose
faster-whisper	1.2.1	worker	GPU accelerated speech transcription
mediapipe	0.10.x	worker	Face Mesh 478 landmark inference
parselmouth	0.4.4	worker	Praat acoustic feature extraction
spacy	3.7.x	worker	NLP preprocessing pipeline
psycopg2-binary	2.9.x	worker, api	PostgreSQL driver
pandas	2.2.x	worker	Time-series data processing
scipy	latest	worker	Statistical computation
celery	5.4.x	worker, api	Distributed task queue
redis	5.x	worker, api	Celery broker transport
fastapi	0.110.x	api	REST API framework
uvicorn	0.29.x	api	ASGI server runtime
pydantic	2.x	worker, api	Schema validation
numpy	1.26.x	worker	Vectorized AU12 calculations
pycryptodome	⩾ 3.20.0	worker	AES-256-GCM encryption
patchright	1.58.2	worker	Stealth browser automation
TikTokLive	5.0.x	worker	TikTok WebSocket ingestion
streamlit	latest	worker	Analytics dashboard

11. Variable extraction matrix

Each analytical variable produced within LSIE-MLF is traceable to its source module, data type, storage location, and extraction methodology.

Variable	Type	Unit/range	Source	Storage	Methodology
Raw Video Frames	`byte stream (H.264)`	N/A	Module A	Transient (kernel buffer)	scrcpy mirroring via USB passthrough (/dev/bus/usb)
Raw Audio PCM	`byte stream (s16le)`	48 kHz mono	Module A	Transient IPC pipe buffer	scrcpy raw audio output
Live Event Payloads	`dict`	N/A	Module B	Transient deque buffer	TikTokLive WebSocket ingestion
Action Combo Trigger	`bool`	True/False	Module B	Persistent Store events table	Composite event correlation
UTC Drift Offset	`float`	seconds	Module C	Transient variable	ADB epoch polling
Resampled Audio Chunks	`byte stream (s16le)`	16 kHz mono	Module C	Transient stdout pipe	FFmpeg resampling pipeline
3D Facial Landmarks	`np.ndarray (478,3)`	normalized coordinates	Module D	Transient memory	MediaPipe Face Mesh
AU12 Intensity Score	`float`	0.0–5.0	Module D	Persistent Store metrics table	IOD normalized geometric ratio
Vocal Pitch F0	`float`	75–600 Hz	Module D	Persistent Store metrics table	Praat pitch extraction
Jitter	`float`	0–1 ratio	Module D	Persistent Store metrics table	Praat jitter algorithm
Shimmer	`float`	0–1 ratio	Module D	Persistent Store metrics table	Praat shimmer algorithm
ASR Transcription	`string`	UTF-8 text	Module D	Persistent Store transcripts table	faster-whisper transcription
Semantic Match	`JSON object`	confidence 0–1	Module D	Persistent Store evaluation table	Azure OpenAI evaluation
Evaluation Variance	`float`	0–infinity	Module E	Persistent Store experiments table	Thompson sampling statistics
External Context Metadata	`JSON dict`	N/A	Module F	Persistent Store context table	patchright web scraping

12. Error handling and recovery specification

LSIE-MLF defines four standardized failure categories. Each module must implement deterministic recovery behavior for each category.

12.1 Network disconnection

Module A	N/A
Module B	Exponential WebSocket reconnection up to 10 retries
Module C	N/A
Module D	Retry Azure OpenAI request once then record null
Module E	Buffer database writes and retry every 5s
Module F	Return empty scrape result

12.2 Hardware device loss

Module A	USB reconnection polling every 2s for 60s then container restart
Module B	N/A
Module C	Freeze drift offset then fallback to zero after timeout
Module D	Terminate worker container if GPU unavailable
Module E	N/A
Module F	N/A

12.3 Worker process crash

Module A	Pipe overflow handled via EAGAIN non-blocking writes
Module B	Reconnect WebSocket on restart
Module C	Restart FFmpeg subprocess automatically
Module D	Docker restart policy on-failure:5
Module E	Log Celery task failure and continue
Module F	Retry scraping task up to two times

12.4 Queue overload

Module A	Kernel pipe overflow discards data silently
Module B	Deque ring buffer evicts oldest entries
Module C	Deque maxlen eviction policy
Module D	Celery queue buffers tasks in Redis
Module E	Overflow records written to CSV fallback storage
Module F	Limit concurrent scraping tasks

13. Autonomous implementation audit checklist

Before deployment the system must satisfy every verification item defined below to ensure deterministic implementation.

#	Audit item	Verification criterion
1	**Directory structure**	Repository directories match the hierarchy defined in Section 3.
2	**Docker topology**	Five containers are defined with correct images, networks, volumes, and restart policies.
3	**IPC lifecycle**	Named pipe lifecycle implemented according to Section 4.A.2.
4	**Audio pipeline**	FFmpeg resampling from 48 kHz to 16 kHz executed correctly.
5	**Drift correction**	ADB polling every 30 seconds with drift_offset applied.
6	**AU12 implementation**	Landmark indices and epsilon guard implemented correctly.
7	**LLM determinism**	Azure OpenAI configured with temperature 0 and structured outputs.
8	**Ephemeral Vault**	AES-256-GCM encryption and 24-hour secure deletion enforced.
9	**Schema validation**	InferenceHandoffPayload validated via Pydantic.
10	**Module contracts**	Inputs, outputs, dependencies, side effects, and failures implemented exactly as specified.
11	**Error handling**	All four failure categories implemented across modules.
12	**Dependency versions**	Pinned runtime versions match the dependency matrix.
13	**Variable traceability**	All variables defined in the extraction matrix are produced by the correct module.
14	**Data classification**	Transient, debug, and permanent storage tiers enforced.
15	**Canonical terminology**	Only canonical component names are used throughout code and configuration.

— END OF SPECIFICATION —
LSIE-MLF Master Technical Specification v2.0 — Production Hardened